Prior as Center: Compact Prediction and Ledger-Anchored Uploads for Heterogeneous Federated Learning

Zixi Lian[1]

Guilin University of Electronic Technology, Guilin 541004, China
Morality026@163.com

Abstract. Federated clients differ in their label distributions, and the updates that carry their models are compressed before they leave the device. We treat these as two halves of one system. For prediction, each client fits a twenty-parameter affine predictor on frozen shared logits and shrinks its bias toward its own class prior rather than toward zero, so the fit keeps the prior where fitted probability mass agrees with local counts and overrides it where it does not. Client-macro NLL falls from 0.680 to 0.645 on CIFAR-10 and from 0.206 to 0.188 on FashionMNIST over three fixed seeds, and in a three-point sweep the mean gain rises from 0.004 to 0.041 on CIFAR-10 as heterogeneity strengthens. For transmission, each three-bit update is bound to a ledger record; an authenticated incorporation receipt ties the encoded payload and its round context to the resulting model state, and a receipt-gated protocol commits the error-feedback residual only after incorporation. Fifteen registrations across five clients and three rounds cost 4.23 M gas, and every resulting model matches the direct path bit for bit.

Keywords: Federated learning; label heterogeneity; compact personalization; ledger anchoring

1 Introduction

A client dominated by one class must use a shared model that also serves nineteen other label distributions. In our label-skewed CIFAR-10 setting, the shared model reaches 38.96% client-macro accuracy, below the 63.06% obtained by predicting from local class frequencies alone. Figure 1 motivates interpreting shared logits locally [1–3].

Fixed unit-strength prior addition uses a preset offset; vector scaling supplies no client-prior center [4, 5]. Fixed addition can duplicate information in shared logits; an unconstrained fit must infer it from limited local labels.

We shrink the bias toward the client's prior instead of adding it. Infinite regularization fixes the prior offset; vanishing regularization removes the penalty. At an optimum, $\lambda(b - \mu) = n - \sum p$: bias deviations track observed counts minus fitted probability mass; agreeing centers are retained and disagreeing ones corrected.

Personalization decides what each client predicts; it does not decide what each client sends. Compressed bytes alone identify neither their client–round context nor the model state that incorporated them. Lost responses, duplicate

delivery and replay can separate upload history from the residual used in the next round. We bind each encoded update to its round context and advance the residual through an authenticated incorporation receipt. We call the resulting system prior-regularized lightweight federated learning (PR-LFL).

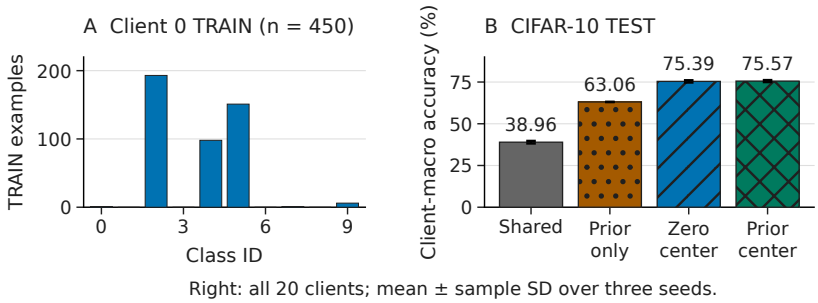


Fig. 1. Local counts expose the prediction gap. Left: CIFAR client 0 TRAIN. Right: client-macro accuracy over twenty clients and three seeds, including prior-only prediction.

Contributions. (i) A twenty-parameter prior-centered predictor addresses label heterogeneity, lowering NLL across all three fixed seeds on both tasks; in this three-point sweep, mean gain increases as heterogeneity strengthens. (ii) Ledger anchoring binds compressed payloads to their round context, and an authenticated incorporation receipt gates error-feedback residual commitment. (iii) Measurements establish 54,067 B codec uploads and 4.23 M gas for fifteen registrations across five clients and three rounds; four fault classes recover the recorded contribution and residual transition.

2 Related Work

Personalization. FedProx, SCAFFOLD and FedNova change shared optimization [6–8]. FedPer/FedRoD retain personal heads; Ditto/pFedMe regularize personal models; FedBN retains local normalization [9–11]. Quantization, sparsification and error feedback reduce traffic [12–14]. PR-LFL combines local prediction with a state-preserving compressed-upload path.

Class priors and calibration. Prior estimation, logit adjustment, Balanced Softmax and FedLC use class counts in prediction or training [15, 16, 4, 17, 18]. PRIOR learns Bregman regularization centers [19]; vector scaling, BCTS and Dirichlet calibration transform scores [5, 20, 21]. LaSCal adapts temperature under label shift [22]; calibration studies motivate uncertainty metrics [23–25]. FedCal and private federated calibration learn scalers, while FedNPC retrains a

server classifier [26–28]. We center a compact frozen-logit fit on each client's observed log class frequencies.

Ledger-anchored federated learning. BlockFL exchanges and verifies local model updates through blockchain coordination [29]. Biscotti combines a ledger of model updates with cryptographic commitments and secure aggregation [30]. We bind each record to the learning state through authenticated incorporation and residual commitment, extending the record beyond a stored hash.

3 Method

3.1 A correctable prior center

For C classes, client i fits its predictor on TRAIN set D_i with frozen shared logits $z(x; w) \in \mathbb{R}^C$; H_i is its disjoint development holdout.

TRAIN counts n_{ic} give add-one priors $\widehat{\pi}_{ic} = (n_{ic}+1)/(|D_i|+C)$ and centered log-priors $\mu_{ic} = \log \widehat{\pi}_{ic} - C^{-1} \sum_k \log \widehat{\pi}_{ik}$. Instead of fixing the offset at $z + \log \widehat{\pi}_i$, we minimize penalized cross-entropy (CE) centered at μ_i:

$$\begin{aligned} \min_{A_i, b_i} & \sum_{(x,y)\in D_i} \mathrm{CE}(A_i \odot z(x; w) + b_i, y) \\ & + \frac{\lambda}{2}\|A_i - \mathbf{1}\|_2^2 + \frac{\lambda}{2}\|b_i - \mu_i\|_2^2, \\ \text{subject to } & A_i \in \mathbb{R}^C, \quad \mathbf{1}^\mathsf{T} b_i = 0. \end{aligned} \tag{1}$$

Vector scaling [5] gives $p_i(x) = \mathrm{softmax}(A_i \odot z(x; w) + b_i)$. Bias centering fixes translation freedom; scales permit ranking changes. At $C = 10$, twenty 32-bit floating-point (FP32) values occupy 80 B.

How the center acts. At the exact minimizer of Eq. 1, the bias satisfies

$$\lambda(b_{ic} - \mu_{ic}) = n_{ic} - \sum_{x\in D_i} p_{ic}(x). \tag{2}$$

Centered biases and zero-sum CE gradients imply a zero constraint multiplier, giving Eq. 2. As $\lambda \to \infty$, $(A_i, b_i) \to (\mathbf{1}, \mu_i)$ recovers fixed prior addition; $\lambda \to 0$ removes regularization but may lack a finite minimizer with missing TRAIN classes. The mean-loss penalty $\lambda/|D_i|$ shrinks smaller clients more strongly. This shrinkage is strongest for the smallest clients, which is where the measured gain concentrates.

For $\lambda > 0$ the objective is strictly convex. We fix $\lambda = 10$, fit TRAIN logits with double-precision (FP64) Newton updates and evaluate stored FP32 states in FP64 (Appendix A, Implementation).

3.2 Low-bit shared training

Shared training uses 20 full-participation rounds, one local epoch, batch 64 and stochastic gradient descent (SGD), learning rate 0.03. Local training minimizes $\mathrm{CE}(z - s_i, y)$ with $s_{ic} = \max(n_{ic}, 1)^{-1/4}$. Error feedback adds a residual before three-bit quantization; aggregation weights decoded updates by TRAIN size [14]. The 136,874-coordinate uniform3 payload occupies 54,067 B within a 54,808 B cap, including scales and its header. Dense downlinks are retained; Section 6 describes the codec, and Appendix A gives the solver and training-loss details.

Matched controls. Zero centering fixes logits, capacity, penalty and solver. Prior-only, fixed-prior and a 650-parameter Linear head isolate counts, correction and capacity. Ridge-10 controls fit bias-only $z + b$, temperature az, or eleven-value BCTS $az + b$ ($a \geq 0.001$) [5, 20]; same-prior controls center b at μ_i. Linear runs one epoch. Frozen FP32 TRAIN logits cost $40|D_i|$ B plus FP64 solver caches.

4 Evaluation Protocol

CIFAR-10 [31] and FashionMNIST [32] use a 136,874-parameter SmallCNN. Fashion is padded to 32×32 and repeated across three channels; normalization is $(x/255-0.5)/0.5$. Classwise Dirichlet-multinomial assignment ($\alpha = 0.1$, twenty clients, minimum 100 examples) yields 37,500/12,500 CIFAR and 45,001/14,999 Fashion TRAIN/holdout examples. A fixed stratified 2,000-example subset monitors training; initialization/batch seeds are paired.

Before TEST, TRAIN counts fix $P(i \mid c) = (n_{ic} + 1)/\sum_j (n_{jc} + 1)$; largest-remainder quotas allocate each class's 1,000 TEST examples. PCG64 orders rows, seeded by $\mathrm{SeedSequence}(20261301, c)$; client index breaks quota ties. Assignments cover TEST once, identically across seeds and methods.

Client-macro NLL/accuracy weight clients equally; tables show three-seed mean ± sample SD.[1] Table 2 adds sample-weighted metrics, client-accuracy decile, Brier score and top-label expected calibration error (ECE), client-macro (c) or pooled (p): 15 equal-width left-closed bins, confidence one in the last.

The heterogeneity sweep retrains the shared model at $\alpha \in \{0.1, 0.5, 1.0\}$ with twenty clients, twenty rounds and three model seeds per task. TRAIN-fitted predictors score disjoint development holdouts excluding the 2,000 monitoring examples. The client-size analysis reuses saved TEST outcomes from the original $\alpha = 0.1$ partition, averaging gain across three seeds per client. We define $\Delta\mathrm{NLL} = \mathrm{NLL}(\mathrm{zero}) - \mathrm{NLL}(\mathrm{prior})$, so positive values indicate improvement.

Cyclic permutation $\widetilde{\mu}_i = \mathrm{roll}(\mu_i, 1)$ preserves center values/norm, penalty, capacity and solver. TRAIN fits and existing correct/zero states score twenty development clients across three seeds.

[1] Sixty client–seed pairs share a partition/models; they are not independent training replicates.

5 Results

5.1 Main results

PR-LFL reaches client-macro NLL 0.645/0.188 (0.6446/0.1880 before rounding) and accuracy 75.57%/92.97% on CIFAR-10/FashionMNIST. Changing the center gives NLL gains of 0.0358/0.0182 and accuracy gains of 0.1777/0.1730 percentage points (Table 1). Learned correction reduces NLL over fixed prior addition by 0.0282/0.0222, placing the prior inside a fitted predictor.

At a matched budget (54,067 B for uniform3, 54,808 B for TopK), a TopK compressor reaches NLL 0.670 and a FedRoD-linear reference 0.677 on CIFAR-10, against 0.645 for uniform3 with the prior center; both are matched in-house implementations over the same three paired seeds. Table 1 shows that the correction also holds under dense FedAvg and FedProx.

Table 1. Prior centering lowers NLL on both tasks and under two dense shared-training paths. Dense FedAvg/FedProx are matched in-house implementations trained on the same partitions and seeds. Client-macro accuracy (%) / NLL, three-seed mean ± sample SD.

	CIFAR-10		FashionMNIST	
Predictor	Acc. (%)	NLL	Acc. (%)	NLL
Shared model	38.96 ± 0.77	1.677 ± 0.029	69.79 ± 1.69	0.749 ± 0.035
Shared + prior	74.60 ± 0.82	0.673 ± 0.014	91.97 ± 0.29	0.210 ± 0.011
Zero vector (20d)	75.39 ± 0.68	0.680 ± 0.014	92.80 ± 0.37	0.206 ± 0.012
Prior BCTS (11d)	75.56 ± 0.52	0.646 ± 0.013	92.84 ± 0.38	0.192 ± 0.011
Prior vector (20d)	75.57 ± 0.60	0.645 ± 0.014	92.97 ± 0.44	0.188 ± 0.011
FedAvg + zero	75.62 ± 0.52	0.679 ± 0.013	92.69 ± 0.41	0.205 ± 0.013
FedProx + zero	75.03 ± 0.41	0.691 ± 0.013	92.62 ± 0.26	0.208 ± 0.013
FedAvg + prior	75.58 ± 0.54	0.644 ± 0.012	92.94 ± 0.41	0.187 ± 0.011
FedProx + prior	74.96 ± 0.59	0.655 ± 0.013	92.84 ± 0.35	0.189 ± 0.011
Local Linear (650d)	74.10 ± 0.34	0.725 ± 0.019	92.22 ± 0.46	0.226 ± 0.018

The prior vector lowers mean NLL and Brier score, whereas same-prior BCTS gives lower mean client ECE (Table 2). Weighted NLL falls by 0.0217/0.0102, compared with the larger client-macro gains, motivating a closer look at how benefit varies across heterogeneous clients.

5.2 Heterogeneity sweep

In this three-point sweep, mean gain increases as heterogeneity strengthens (Fig. 2a). As α decreases from 1.0 through 0.5 to 0.1, gain rises from 0.0042 through 0.0101 to 0.0408 on CIFAR-10 and from 0.0024 through 0.0047 to 0.0132 on FashionMNIST. Each α uses one partition realization and three model seeds. At $\alpha = 0.1$, development NLL improves in 60/60 CIFAR and 59/60 Fashion client–seed pairs, concentrating the largest mean benefit at the strongest heterogeneity measured in this sweep.

Table 2. Prior-centered vector scaling lowers NLL and Brier score; BCTS gives lower client ECE. Upper: Brier/ECE (lower is better), ECE in percentage points. P denotes prior centering. Lower: means; accuracy is a proportion.

	CIFAR-10			FashionMNIST		
Predictor	Brier	ECE_c	ECE_p	Brier	ECE_c	ECE_p
Zero	0.343 ± 0.008	6.15 ± 0.20	1.35 ± 0.41	0.108 ± 0.006	3.69 ± 0.45	0.58 ± 0.20
P-BCTS	0.341 ± 0.007	5.37 ± 0.23	1.30 ± 0.10	0.105 ± 0.005	2.82 ± 0.07	0.45 ± 0.21
P-vector	0.340 ± 0.008	5.48 ± 0.17	1.49 ± 0.24	0.103 ± 0.006	3.05 ± 0.18	0.47 ± 0.20

	CIFAR-10		FashionMNIST	
Endpoint	Zero	Prior	Zero	Prior
Weighted NLL	0.6788	0.6571	0.1588	0.1486
Weighted accuracy	0.7475	0.7471	0.9462	0.9473
Client accuracy q_{10}	0.6027	0.6065	0.8578	0.8566

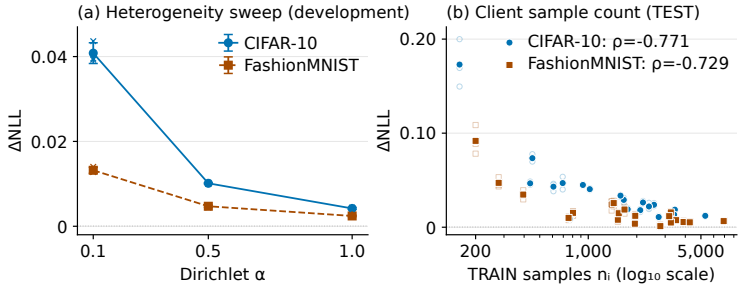


Fig. 2. Mean gain is larger under stronger heterogeneity across the three-point sweep. (a) Development gain versus α: three-seed means, sample SD and seed markers. (b) Saved TEST gain versus client TRAIN sample count n_i (log axis): twenty client means per task and seed observations. Positive $\Delta\text{NLL} = \text{NLL(zero)} - \text{NLL(prior)}$ favors prior centering.

5.3 Who benefits: client sample count

Gain is negatively associated with client TRAIN sample count, with Spearman $\rho = -0.771/-0.729$ on CIFAR-10/FashionMNIST (Fig. 2b). Each client mean averages three seeds within the original fixed partition. This association places the measured gain among smaller clients, where the mean-loss penalty $\lambda/|D_i|$ gives the prior center greater weight.

5.4 Class-alignment control

Permuting class labels in the prior raises development NLL by 0.071/0.044 on CIFAR-10/FashionMNIST at fixed center norm and predictor capacity, attributing the benefit to class alignment.

6 Anchored Uploads

The compact predictor interprets a shared model through the client's label distribution. That model is built from updates whose quantization error persists across rounds, so aggregation and residual commitment must agree on which encoded update was incorporated. We bind each compressed update to a ledger record and advance its residual through an authenticated incorporation receipt, connecting message delivery to the state used by the next training round.

6.1 Bind the encoded update to its training context

The registry is not a database replica. The coordinator writes its local table and selects contributions, so that table alone offers little acceptance evidence. Ledger registration provides ordered, transaction-addressable evidence outside that database write path to participants with ledger access. An authenticated receipt links the record to the next residual's model state, separating ledger registration checks from receipt-based incorporation checks.

For client i in round t, let δ_i^t be the local model update and r_i^t the committed error-feedback residual. The client encodes $u_i^t = \delta_i^t + r_i^t$ into bytes B_i^t, decodes $q_i^t = Q(B_i^t)$, and stages $\widetilde{r}_i^{t+1} = u_i^t - q_i^t$. Uniform3 represents each block of 256 coordinates with three-bit codes, a width byte and an FP32 scale, preceded by a 64-byte header. Persisting the encoded bytes together with the staged residual makes every retry refer to the same numerical update.

An upload envelope carries the activity, client, round, record identifier, nonce, base-model digest, parameter-layout identifier, codec-specification digest and payload. With a per-client symmetric key shared by the client and coordinator, HMAC-SM3 authenticates the length-prefixed envelope; its SM3 digest identifies the authenticated content [33, 34]. Table 3 separates these transmitted bytes from their chain binding, making each record resolve to a precise round and decoder.

Table 3. The record binds transmitted context to ledger evidence and local learning state.

Role	Application / local state	Registry / receipt evidence
Identity and freshness	Activity, client, round, record, nonce	Digests of activity/client/nonce; round; submitting account
Model and decoder	Base-model digest, layout, codec and specification	Base-model, layout and specification digests
Update content	Exact compressed payload and HMAC-SM3 tag	Authenticated-envelope digest and inclusion block
Incorporation	Pending bytes, staged residual and generation	Authenticated session/challenge, state and result-model digest

The registry maps each client identity to an authorized account. A registration stores the envelope digest and round context, together with the submitting account and inclusion block; record, nonce and client–round checks reject conflicting bindings. The bridge retrieves the contract state, registration event, successful transaction and canonical inclusion block, then checks the configured chain, contract, uploader and two descendant blocks. This readback links the accepted envelope to an executed registration before aggregation begins.

6.2 Advance residuals through incorporation receipts

The coordinator checks envelope authentication and the persisted round context before decoding an update. It then records the confirmed binding, accepts the decoded contribution and finalizes the fixed participant set with sample-count weights. Finalization writes the resulting model and incorporated record states in one database transaction; a subsequent receipt includes the envelope context, session, request challenge, incorporation state and resulting-model digest. The receipt connects the accepted upload to the model state that authorizes the client's residual transition.

The client checks the receipt MAC, session, challenge, envelope binding and included chain evidence before committing the staged residual. The state transition is

$$(r_i^t, \mathrm{pending}(B_i^t, \widetilde{r}_i^{t+1})) \xrightarrow{\text{verified incorporation}} (\widetilde{r}_i^{t+1}, \mathrm{committed}), \tag{3}$$

with the local generation increment and pending-state update written atomically. Until this transition, the previous residual and exact pending bytes remain available. Figure 3 places model incorporation between chain registration and residual commitment, preserving the order required by error feedback.

6.3 Recover the same update after an interrupted exchange

A client reconstructs a retry from its persisted pending envelope. When registration succeeded before the response was lost, the coordinator retrieves that binding and continues acceptance; when aggregation finished before the response was lost, the coordinator returns the incorporated state and resulting-model digest. Reopening the service and client stores therefore resumes the existing record through the same receipt path.

Repeated acceptance, round finalization and residual commitment check their persisted states before applying a transition. While a round awaits its participant set, the coordinator retains the open round and model while clients retain pending updates and previous residuals. Recovery thus preserves one contribution and one residual-generation change per client–round, giving the upload history a direct interpretation in the learning state.

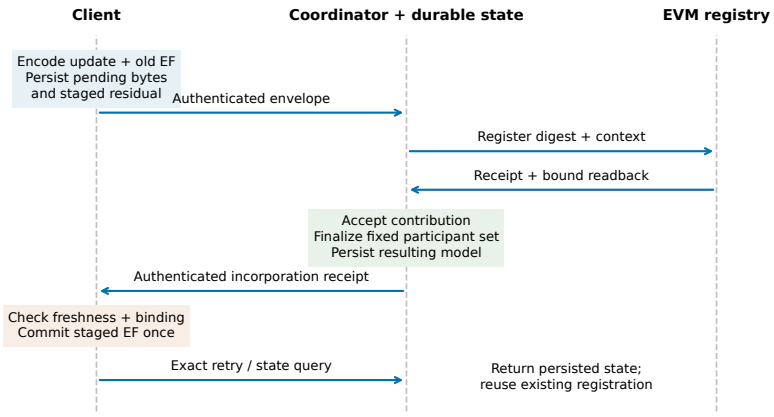


Fig. 3. An incorporation receipt connects chain registration to the next residual. Retries reuse persisted bytes and recover the recorded state before the client advances its generation.

7 Upload Evaluation

The predictive experiments in Section 5 apply the same three-bit codec; the integration runs here add the ledger and receipt path on top of it. We evaluate the upload path with real CIFAR-10 TRAIN updates, loopback HTTP, SQLite state and the Solidity registry on a local EVM development chain. Two integration runs (two clients over two rounds; five clients over three rounds) share the same measurement boundaries and recovery protocol. This pairing tests the transition from a compact encoded update to a recoverable contribution in the shared model.

7.1 Compact messages retain their bound context

For the 136,874-parameter model, uniform3 produces 54,067 B including block scales and the codec header, compared with 547,560 B for the dense message profile. The authenticated envelope adds 341 B, giving 54,408 B per fresh upload. The two runs carry 217,632 B across four uploads and 816,120 B across fifteen uploads; the unchanged message size keeps the context cost explicit as the workload grows.

Table 4 separates message bytes, registration gas and verification work. The codec reduces the dense upload profile by 90.13%, while the authenticated message occupies 9.94% of that dense profile. Registration calldata measures 356 B per transaction, and signed transactions occupy 472–473 B in the five-client run. Keeping application messages and ledger transactions as separate quantities exposes the cost of anchoring alongside the compression saving.

Table 4. Anchoring preserves the 54,408 B application message as the activity grows. Registration gas excludes contract deployment; receipt verification measures local authentication and bound chain fields.

Metric	2×2	5×3
Fresh uploads	4	15
Codec payload / upload (B)	54,067	54,067
Authenticated message / upload (B)	54,408	54,408
Total fresh message bytes	217,632	816,120
Registration gas, total	1,140,380	4,229,478
Registration gas / upload, mean	285,095.0	281,965.2
Receipt verification, mean (ms)	0.457	0.380
Ledger readback, mean (ms)	17.912	15.573

7.2 Fifteen registrations preserve the update trajectory

The five-client run uses original clients 0–4 with 450, 159, 1,661, 3,446 and 2,558 TRAIN examples, respectively. Each round runs one local epoch with batch size 64 and learning rate 0.03, yielding 393 gradient evaluations over three rounds. Every computed local update feeds independently encoded direct and protocol paths; FP32 training and coding are followed by FP64 protocol state and deterministic weighted aggregation. After every round, the resulting model and committed residuals match the direct path bit for bit, connecting all fifteen anchored uploads to the same learning trajectory.

The pinned runtime uses Ganache 7.9.2, Solidity 0.8.30 and ethers 6.15.0. Fifteen registrations consume 4,229,478 gas in total, with a mean of 281,965.2 gas per upload. The matched four-registration run consumes 1,140,380 gas, with a mean of 285,095.0 per upload. Figure 4 presents these measured configurations with their transaction counts, giving the aggregate cost a concrete workload basis.

Local verification of the first successful commit-path incorporation receipt averages 0.380 ms across fifteen client–round records, compared with 0.457 ms across four records in the smaller run. This timing includes JSON parsing, HMAC-SM3, freshness, context and embedded chain-field checks; ledger retrieval is timed separately and averages 15.573 ms in the five-client run. Separating the two measurements identifies the work that authorizes the residual transition after the ledger evidence has arrived.

7.3 Interrupted responses recover one contribution

Both runs exercise lost responses after chain registration and after model finalization, reopen service and client databases, and retry the exact pending bytes. Queries recover authenticated incorporated states, while repeated uploads, finalization and residual commits retain the recorded model and generation. Table 5 groups these cases by the interrupted transition, showing how recovery carries the same update through completion.

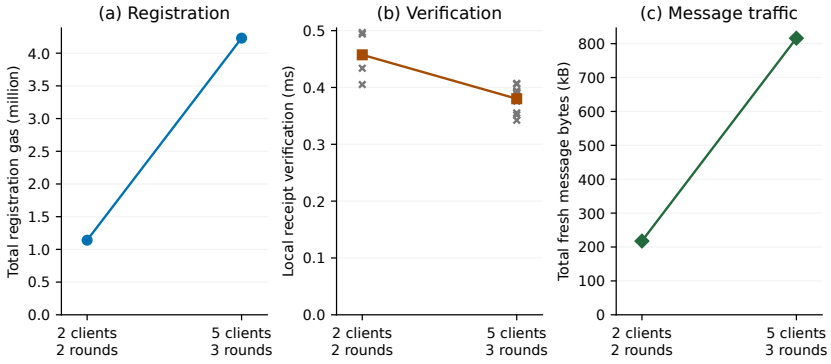


Fig. 4. Measured cost for four and fifteen fresh uploads. Registration gas and message traffic accumulate over the displayed client–round workloads; crosses show individual local receipt-verification times and squares their means.

Table 5. Recovery resumes the recorded client–round transition. All four classes pass in both measured activities.

Injected interruption	Recovered state and action	Result
Reply lost after registration	Read back binding; accept the pending bytes	Pass
Reply lost after finalization	Query incorporated record; commit staged residual	Pass
Service and database reopen	Restore pending message and persisted model state	Pass
Duplicate upload, finalize or commit	Reuse recorded contribution and generation	Pass

Payload mutations, signed context mismatches, altered receipts and stale challenges are rejected before authorizing the corresponding state transition. In the five-client run, each round finalizes once and each client generation advances once per round, while recovery adds zero encoding calls, registration requests or model contributions. These outcomes make the authenticated record a bridge between compressed transmission and the state used by subsequent learning.

8 Discussion

Three observations explain why 20 parameters close this much of the gap. Counts supply the center, fitting corrects it, and the zero-center control isolates information from capacity. A 650-parameter Linear control uses over 32 times as many values but reaches higher NLL, while eleven-value BCTS with the same center approaches vector scaling with lower client ECE. The center, rather than the parameterization, carries the gain.

The upload half answers a different question: whether a contribution can be shown to have entered the model. A local table would store the same fields, but it would be written by the party that also decides which updates are aggregated, so the client's own record would prove little. Placing the binding on a ledger moves it outside the coordinator's local database write path, and an incorporation receipt issued by the coordinator ties the on-chain entry to the model state that authorizes the next residual. Retries then resume from a recorded state rather than from an assumption.

The accounting separates the costs that deployments trade differently. Personalization adds 80 B locally, one TRAIN forward pass and a twenty-dimensional convex solve. Shared training requires 54,067 B uploads and a 547,496 B FP32 residual per client, with dense downlinks. Anchoring adds 341 B of bound context and authentication per upload and one registration per client–round, measured at 281,965 gas on average across fifteen registrations.

9 Conclusion

PR-LFL makes the client's class prior a regularization center that the data can override, and binds the compressed updates that carry the shared model to records that can be checked after the fact. Twenty parameters and 80 B of local state lower client-macro NLL from 0.680 to 0.645 on CIFAR-10 and from 0.206 to 0.188 on FashionMNIST, and the mean gain rises as heterogeneity strengthens across a three-point sweep. Three-bit uploads cost 54,067 B, 90.1% below dense; fifteen registrations across five clients and three rounds cost 4.23 M gas, and every resulting model matches the direct path bit for bit.

A Implementation Details

FP64 Newton fits store FP32 states. Training uses $\mathrm{CE}(z - s_i, y)$, with $s_{ic} = \max(n_{ic}, 1)^{-1/4}$ [18].

References

1. McMahan, B., Moore, E., Ramage, D., Hampson, S., Arcas, B.A.y.: Communication-efficient learning of deep networks from decentralized data. In: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics, *Proceedings of Machine Learning Research*, vol. 54, pp. 1273–1282. PMLR (2017)
2. Arivazhagan, M.G., Aggarwal, V., Singh, A.K., Choudhary, S.: Federated learning with personalization layers. arXiv preprint arXiv:1912.00818 (2019). DOI 10.485 50/arXiv.1912.00818
3. Chen, H.Y., Chao, W.L.: On bridging generic and personalized federated learning for image classification. In: International Conference on Learning Representations (2022)

4. Menon, A.K., Jayasumana, S., Rawat, A.S., Jain, H., Veit, A., Kumar, S.: Long-tail learning via logit adjustment. arXiv preprint arXiv:2007.07314v2 (2021)
5. Guo, C., Pleiss, G., Sun, Y., Weinberger, K.Q.: On calibration of modern neural networks. In: Proceedings of the 34th International Conference on Machine Learning, *Proceedings of Machine Learning Research*, vol. 70, pp. 1321–1330. PMLR (2017)
6. Li, T., Sahu, A.K., Zaheer, M., Sanjabi, M., Talwalkar, A., Smith, V.: Federated optimization in heterogeneous networks. In: Proceedings of Machine Learning and Systems (2020)
7. Karimireddy, S.P., Kale, S., Mohri, M., Reddi, S., Stich, S., Suresh, A.T.: SCAFFOLD: Stochastic controlled averaging for federated learning. In: Proceedings of the 37th International Conference on Machine Learning, *Proceedings of Machine Learning Research*, vol. 119, pp. 5132–5143. PMLR (2020)
8. Wang, J., Liu, Q., Liang, H., Joshi, G., Poor, H.V.: Tackling the objective inconsistency problem in heterogeneous federated optimization. In: Advances in Neural Information Processing Systems (2020)
9. Li, T., Hu, S., Beirami, A., Smith, V.: Ditto: Fair and robust federated learning through personalization. In: International Conference on Machine Learning, *Proceedings of Machine Learning Research*, vol. 139, pp. 6357–6368. PMLR (2021)
10. Dinh, C.T., Tran, N., Nguyen, J.: Personalized federated learning with moreau envelopes. In: Advances in Neural Information Processing Systems, vol. 33, pp. 21,394–21,405 (2020)
11. Li, X., Jiang, M., Zhang, X., Kamp, M., Dou, Q.: Fedbn: Federated learning on non-iid features via local batch normalization. In: International Conference on Learning Representations (2021)
12. Alistarh, D., Grubic, D., Li, J., Tomioka, R., Vojnovic, M.: QSGD: Communication-efficient SGD via gradient quantization and encoding. In: Advances in Neural Information Processing Systems, vol. 30, pp. 1709–1720 (2017)
13. Stich, S.U., Cordonnier, J.B., Jaggi, M.: Sparsified SGD with memory. In: Advances in Neural Information Processing Systems, vol. 31 (2018)
14. Karimireddy, S.P., Rebjock, Q., Stich, S., Jaggi, M.: Error feedback fixes SignSGD and other gradient compression schemes. In: Proceedings of the 36th International Conference on Machine Learning, *Proceedings of Machine Learning Research*, vol. 97, pp. 3252–3261. PMLR (2019)
15. Saerens, M., Latinne, P., Decaestecker, C.: Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure. Neural Computation **14**(1), 21–41 (2002). DOI 10.1162/089976602753284446
16. Lipton, Z., Wang, Y.X., Smola, A.: Detecting and correcting for label shift with black box predictors. In: International Conference on Machine Learning, *Proceedings of Machine Learning Research*, vol. 80, pp. 3122–3130. PMLR (2018)
17. Ren, J., Yu, C., Sheng, S., Ma, X., Zhao, H., Yi, S., Li, H.: Balanced meta-softmax for long-tailed visual recognition. In: Advances in Neural Information Processing Systems, vol. 33, pp. 4175–4186 (2020)
18. Zhang, J., Li, Z., Li, B., Xu, J., Wu, S., Ding, S., Wu, C.: Federated learning with label distribution skew via logits calibration. In: Proceedings of the 39th International Conference on Machine Learning, *Proceedings of Machine Learning Research*, vol. 162, pp. 26,311–26,329. PMLR (2022)
19. Shi, M., Zhou, Y., Wang, K., Zhang, H., Huang, S., Ye, Q., Lv, J.: PRIOR: Personalized prior for reactivating the information overlooked in federated learning. In: Advances in Neural Information Processing Systems, vol. 36, pp. 28,378–28,392 (2023). DOI 10.52202/075280-1234

20. Alexandari, A.M., Kundaje, A., Shrikumar, A.: Maximum likelihood with bias-corrected calibration is hard-to-beat at label shift adaptation. arXiv preprint arXiv:1901.06852v5 (2020)
21. Kull, M., Perello Nieto, M., Kängsepp, M., Silva Filho, T., Song, H., Flach, P.: Beyond temperature scaling: Obtaining well-calibrated multi-class probabilities with dirichlet calibration. In: Advances in Neural Information Processing Systems, vol. 32 (2019)
22. Popordanoska, T., Radevski, G., Tuytelaars, T., Blaschko, M.B.: LaSCal: Label-shift calibration without target labels. In: Advances in Neural Information Processing Systems, vol. 37, pp. 65,386–65,414 (2024). DOI 10.52202/079017-2088
23. Vaicenavicius, J., Widmann, D., Andersson, C., Lindsten, F., Roll, J., Schön, T.: Evaluating model calibration in classification. In: The 22nd International Conference on Artificial Intelligence and Statistics, *Proceedings of Machine Learning Research*, vol. 89, pp. 3459–3467. PMLR (2019)
24. Kumar, A., Liang, P.S., Ma, T.: Verified uncertainty calibration. In: Advances in Neural Information Processing Systems, vol. 32 (2019)
25. Ovadia, Y., Fertig, E., Ren, J., Nado, Z., Sculley, D., Nowozin, S., Dillon, J., Lakshminarayanan, B., Snoek, J.: Can you trust your model's uncertainty? evaluating predictive uncertainty under dataset shift. In: Advances in Neural Information Processing Systems, vol. 32 (2019)
26. Peng, H., Yu, H., Tang, X., Li, X.: FedCal: Achieving local and global calibration in federated learning via aggregated parameterized scaler. arXiv preprint arXiv:2405.15458 (2024)
27. Maddock, S., Cormode, G., Maple, C.: Private federated multiclass post-hoc calibration. arXiv preprint arXiv:2510.01987 (2025)
28. Gao, J., Zhao, H., Yang, Y., Guo, D.: FedNPC: Stochastic noise-driven post-hoc classifier calibration method for federated long-tailed learning. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition Findings, pp. 7737–7746 (2026)
29. Kim, H., Park, J., Bennis, M., Kim, S.L.: Blockchained on-device federated learning. IEEE Communications Letters **24**(6), 1279–1283 (2020). DOI 10.1109/LCOMM.2019.2921755
30. Shayan, M., Fung, C., Yoon, C.J.M., Beschastnikh, I.: Biscotti: A blockchain system for private and secure federated learning. IEEE Transactions on Parallel and Distributed Systems **32**(7), 1513–1525 (2021). DOI 10.1109/TPDS.2020.3044223
31. Krizhevsky, A.: Learning multiple layers of features from tiny images. Technical report (2009)
32. Xiao, H., Rasul, K., Vollgraf, R.: Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms. arXiv preprint arXiv:1708.07747 (2017). DOI 10.48550/arXiv.1708.07747
33. Krawczyk, H., Bellare, M., Canetti, R.: HMAC: Keyed-hashing for message authentication. RFC 2104, RFC Editor (1997). DOI 10.17487/RFC2104. Informational
34. Shen, S., Lee, X., Tse, R.H., Wong, W.K., Yang, P.Y.: The SM3 cryptographic hash function. Internet-Draft draft-sca-cfrg-sm3-02, Internet Engineering Task Force (2018). Expired Internet-Draft; work in progress; not an RFC